COMP2048: Theory of Computing

Dr. Kasra Khosoussi

Week 3

12/3/2026

Recap

- Regular languages
- Closure properties of regular languages
- Product construction (closure under union & intersection)
- Regular expressions
- Non-deterministic Finite Automata (NFA)

Today's Plan

- Non-deterministic Finite Automata (NFA)
- Revisit closure properties
- Converting NFA to DFA (Subset Construction)
- Converting regex to NFA
- Pumping Lemma for Regular Languages

Non-determinism

In DFA:

- From each state and for each symbol there is **exactly one** transition
- Each transition consumes a symbol

What if we allow:

- zero, one, or more possible next states in each transition
- transitions without consuming input

$\Rightarrow$ can be in **multiple states** after reading kth symbol from input

does non-determinism make our model more powerful?

does non-determinism make our model more powerful?[a]

[a] i.e., can we recognize a language we couldn't recognize before with DFAs?

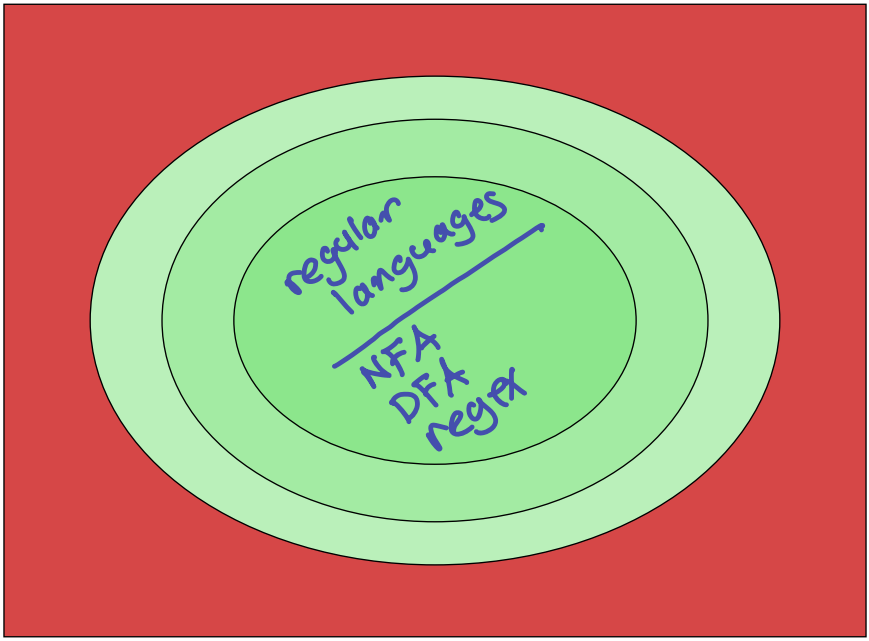
Theorem.

set of languages recognized by **DFAs**
=
set of languages recognized by **NFAs**
=
set of languages described by **regex**
=
Regular Languages

Proof idea: Proof by construction; i.e., for any DFA/NFA/regex, we can algorithmically construct a DFA/NFA/regex that recognizes the same language

$\Rightarrow L$ is regular $\Leftrightarrow \exists$ a DFA/NFA/regex that recognizes L

Our Big Picture Revisited



Rectangle: set of all decision problems (or, languages) over Σ

DFA vs. NFA

- Usually it is more natural to design an NFA
- An NFA can be **exponentially smaller** than an equivalent DFA
- In the worst case, converting an n-state NFA to a DFA may produce up to 2^n states (via the subset construction)
- Intuition: NFA can be in multiple states at the same time
- It is easier to convert regex to NFA
- In practice, NFAs are often used as a theoretical tool/convenient design model and then converted to an equivalent DFA for execution

  e.g., regex $\rightarrow$ NFA $\rightarrow$ DFA (for implementation)

Converting NFA[1] to DFA via Subset Construction

Given an NFA N, construct a DFA M such that $L(M) = L(N)$

$$N = (Q, \Sigma, \delta, q_0, F) \quad M = (Q', \Sigma, \delta', q_0', F')$$

Idea:

[1] Assuming there are **no** ε **transitions** — see Sipser's book for the general case.

Converting NFA[1] to DFA via Subset Construction

Given an NFA N, construct a DFA M such that $L(M) = L(N)$

$$N = (Q, \Sigma, \delta, q_0, F) \quad M = (Q', \Sigma, \delta', q_0', F')$$

Idea:

- Each state of DFA M corresponds to **a subset of NFA states**

$$Q' = \mathcal{P}(Q) \triangleq \{R \mid R \subseteq Q\} \quad \text{(power set)}$$

defined as

(2^Q)

[1] Assuming there are **no** ε **transitions** — see Sipser's book for the general case.

Converting NFA[1] to DFA via Subset Construction

Given an NFA N, construct a DFA M such that $L(M) = L(N)$

$$N = (Q, \Sigma, \delta, q_0, F) \quad M = (Q', \Sigma, \delta', q_0', F')$$

Idea:

- Each state of DFA M corresponds to **a subset of NFA states**

$$Q' = \mathcal{P}(Q) \triangleq \{R \mid R \subseteq Q\} \quad \text{(power set)}$$

- Consider all possible transitions; i.e., for any $R \in Q'$ and $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

union

[1] Assuming there are **no** ε **transitions** — see Sipser's book for the general case.

Converting NFA[1] to DFA via Subset Construction

Given an NFA N, construct a DFA M such that $L(M) = L(N)$

$$N = (Q, \Sigma, \delta, q_0, F) \quad M = (Q', \Sigma, \delta', q_0', F')$$

Idea:

- Each state of DFA M corresponds to **a subset of NFA states**

$$Q' = \mathcal{P}(Q) \triangleq \{R \mid R \subseteq Q\} \quad \text{(power set)}$$

- Consider all possible transitions; i.e., for any $R \in Q'$ and $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

- Start state q_0':

$$q_0' = \{q_0\}$$

[1] Assuming there are **no** ε **transitions** — see Sipser's book for the general case.

Converting NFA[1] to DFA via Subset Construction

Given an NFA N, construct a DFA M such that $L(M) = L(N)$

$$N = (Q, \Sigma, \delta, q_0, F) \quad M = (Q', \Sigma, \delta', q_0', F')$$

Idea:

- Each state of DFA M corresponds to **a subset of NFA states**

$$Q' = \mathcal{P}(Q) \triangleq \{R \mid R \subseteq Q\} \quad \text{(power set)}$$

- Consider all possible transitions; i.e., for any $R \in Q'$ and $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

- Start state q_0':

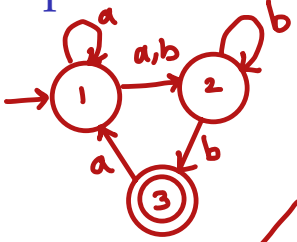
$$q_0' = \{q_0\}$$

- Accepting states:

$$F' = \{R \in Q' \mid R \text{ contains an accept state of } N\}$$

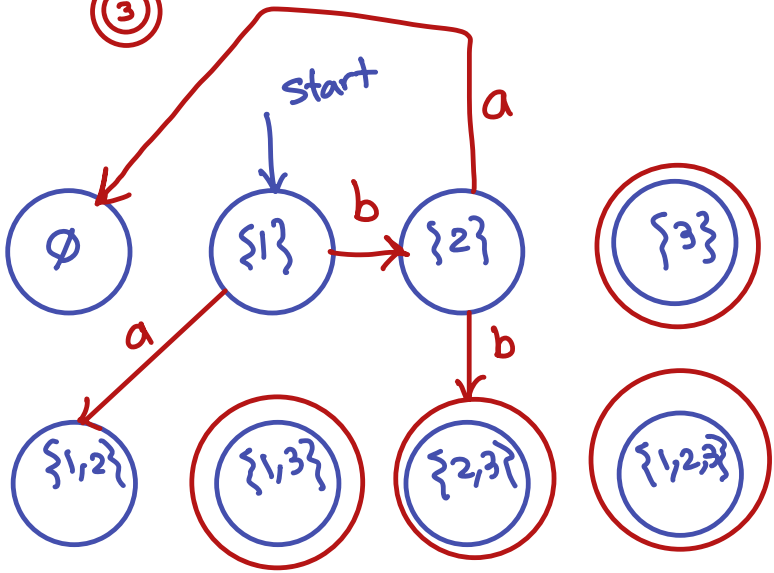
[1] Assuming there are **no** ε **transitions** — see Sipser's book for the general case.

Example



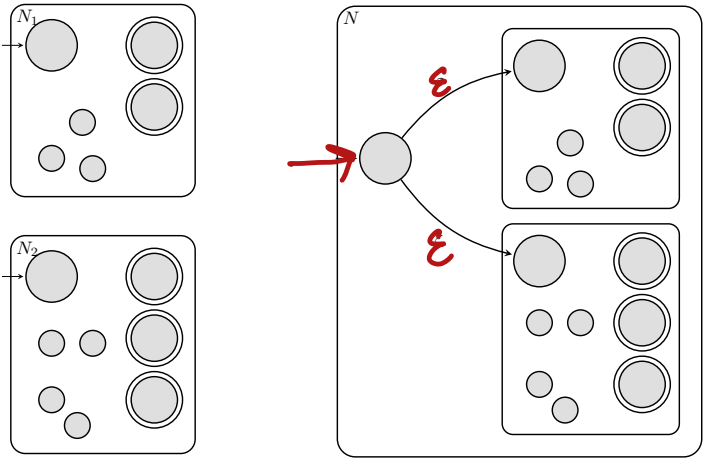
$Q = \{1, 2, 3\}$

$Q'_{DFA} = P(Q) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$



NFA for Union of Regular Languages

Finite automata (DFA/NFA) N_1 and N_2 recognizing L_1 and L_2

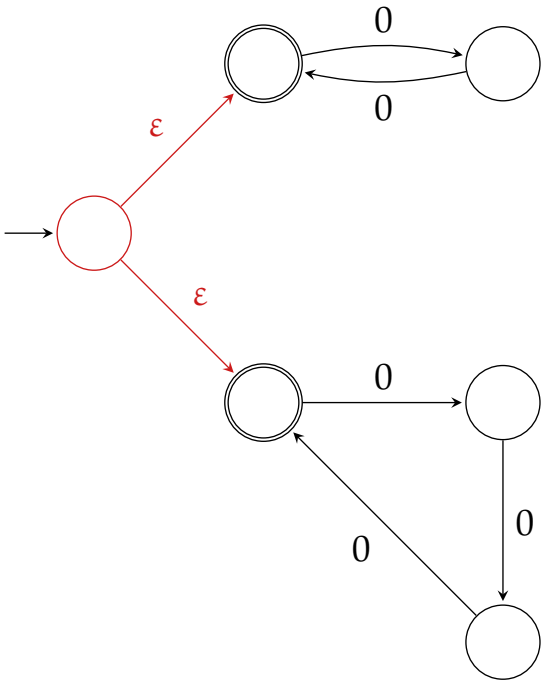


Why it works

On input w, the new NFA *non-deterministically* chooses to simulate either N_1 or N_2. Therefore it accepts exactly when $w \in L_1 \cup L_2$.

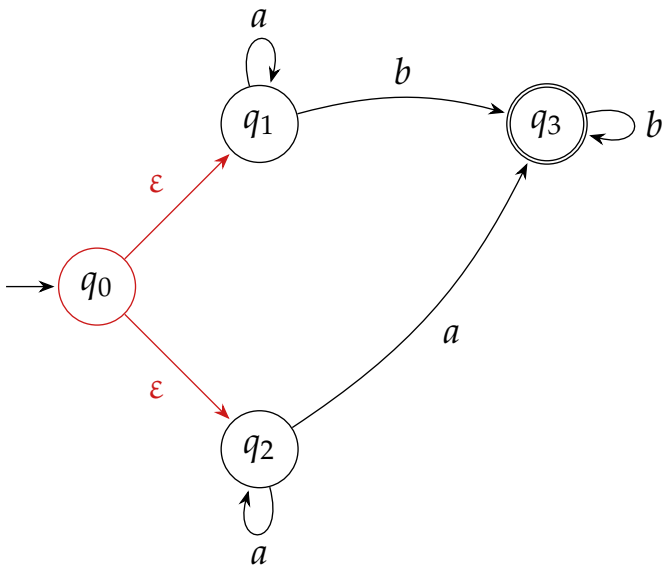
Example

$\Sigma = \{0\}$



Example

$\Sigma = \{a, b\}$



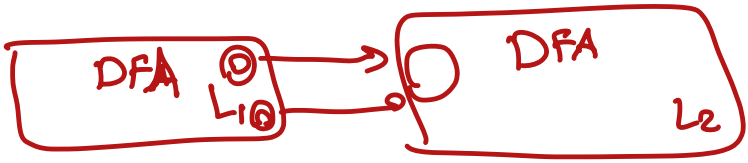
NFA for Concatenation

- Concatenating two regular languages produces a regular language

$$L_1 L_2 \triangleq \{w \in \Sigma^* \mid w = xy \text{ such that } x \in L_1, y \in L_2\}$$

- But we didn't prove it (i.e., we didn't construct a DFA for $L_1 L_2$)

How should we do this?



NFA for Concatenation

- Concatenating two regular languages produces a regular language

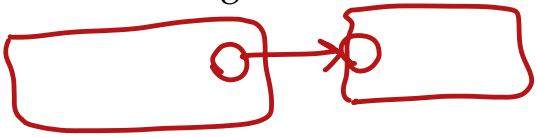
$$L_1 L_2 \triangleq \{w \in \Sigma^* \mid w = xy \text{ such that } x \in L_1, y \in L_2\}$$

- But we didn't prove it (i.e., we didn't construct a DFA for $L_1 L_2$)

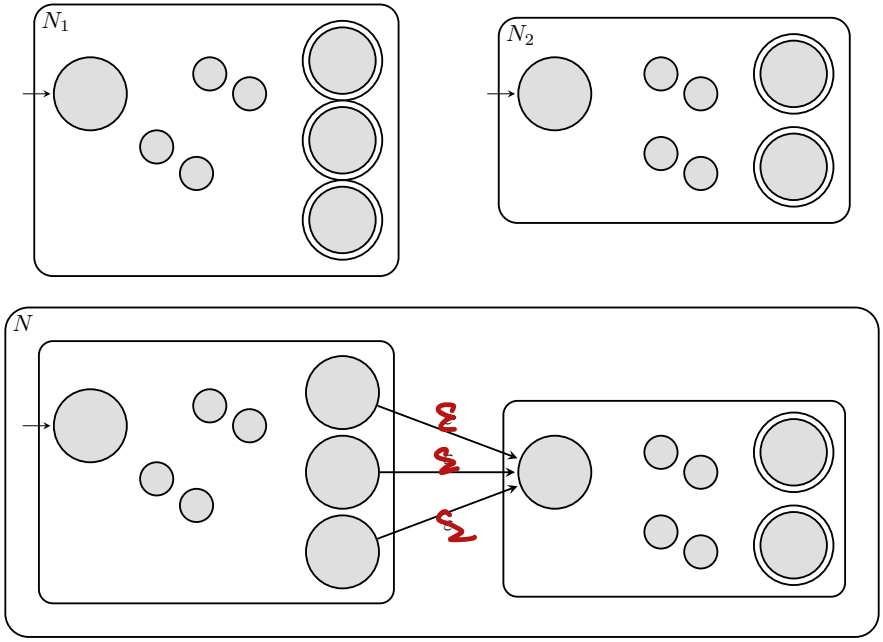
How should we do this?

- **Idea #1:** Put the two DFAs in series? Not exactly

  Challenge: we *a priori* don't know where to split the input w into $w = xy$ such that $x \in L_1$ and $y \in L_2$. For example, the automaton for L_1 may enter an accept state at some point during processing of w, but leave it for other states and come back to an accept state again before switching to the automaton for L_2!

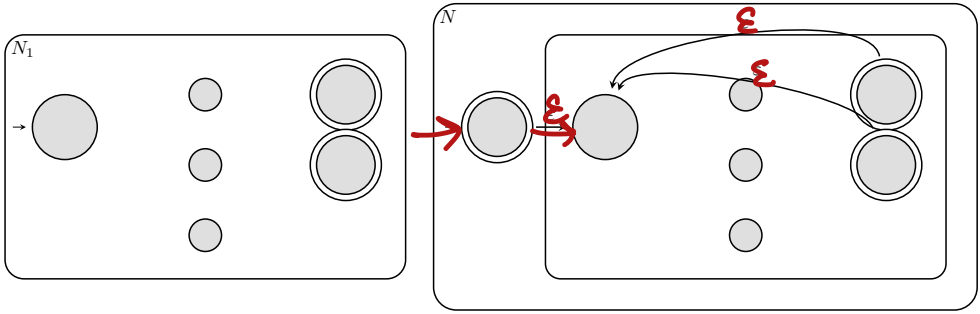


NFA for Concatenation



NFA for Star[2]

$$L^* \triangleq \{\varepsilon\} \cup L \cup L^2 \cup L^3 \cup \ldots$$



[2] $L^k \triangleq L \circ L \circ \ldots \circ L$ (k times) — i.e., k times concatenation of L

Converting Regular Expression to NFA

For each form of regular expression we construct an equivalent NFA.

Base cases:

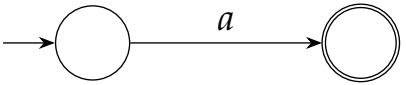
- $\emptyset$
- ε
- $a \in \Sigma$

Recursive cases:

- $R \cup S$
- RS
- R^*

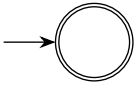
NFA for Base Cases

Regular expression: a



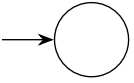
$$L = \{a\}$$

Regular expression: ε



$$L = \{\varepsilon\}$$

Regular expression: $\emptyset$

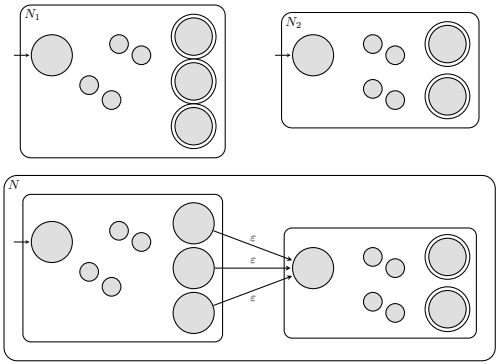


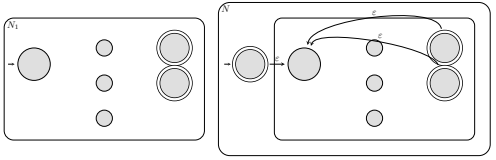
$$L = \emptyset$$

NFA for Recursive Cases

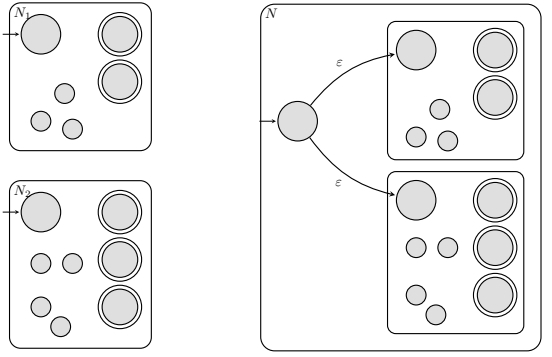
Regular Operations

Concat.





Star

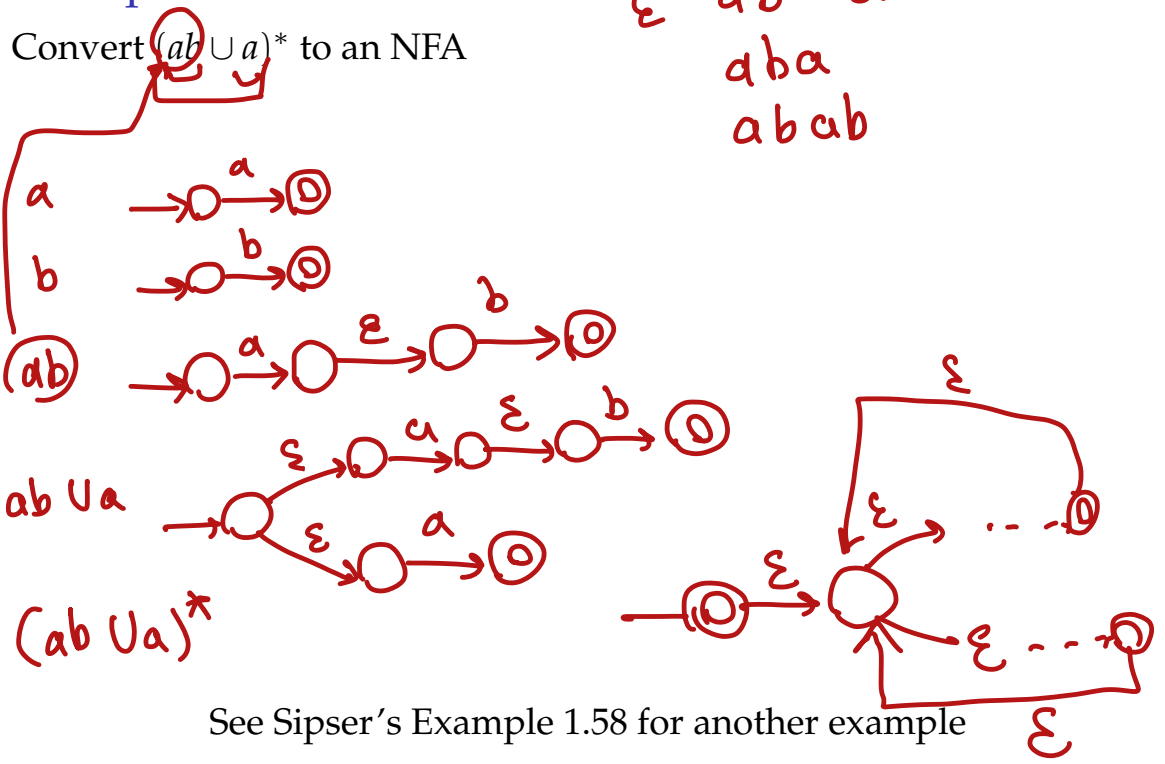


Union

use constructions for union, concatenation, and star

Example

Convert $(ab \cup a)^*$ to an NFA



See Sipser's Example 1.58 for another example

Beyond Regular Languages

$$L = \{1^n 0^n \mid n \geqslant 0\}$$

- We know how to show a language is regular

  construct a DFA (or NFA/regex) that recognizes it

- But how to **prove** a language is not regular?

  i.e., show that there is no DFA that recognizes the language

- Maybe L is regular but the DFA that recognizes it has 1M states!

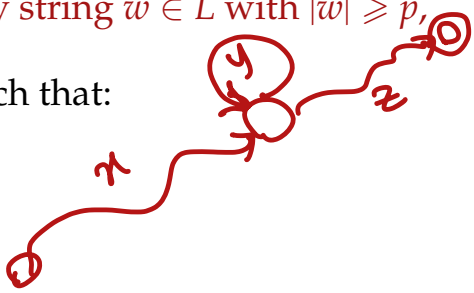
Pumping Lemma for Regular Languages

Theorem (Pumping Lemma):
If L is a regular language, then there exists a constant $p \geqslant 1$ (the pumping length) such that:

For every string $w \in L$ with $|w| \geqslant p$,

there exist strings x, y, z such that:

- $w = xyz$
- $|y| \geqslant 1$
- $|xy| \leqslant p$
- For all $i \geqslant 0$, the string $xy^i z \in L$



Idea: In a long enough string, a finite automaton must repeat states – this creates a loop that can be pumped.

Using the Pumping Lemma to Prove Non-Regularity

1. Assume, for contradiction, that L is regular
2. Then Pumping Lemma applies: there exists a pumping length p
3. Choose a string $w \in L$ such that[3]

$$|w| \geqslant p$$

4. Consider all possible decompositions[4]

$$w = xyz$$

such that

$$|xy| \leqslant p \qquad |y| \geqslant 1$$

5. Show that for some $i \geqslant 0$,

$$xy^i z \notin L$$

contradicts Pumping Lemma $\Rightarrow$ L is *not* regular

[3] Need to be clever about the string we pick so that it leads to contradiction down the line! Not all choices lead to contradiction!

[4] Note that the lemma states that there **exist a** decomposition $w = xyz$ — so we need to show all such decompositions lead to contradiction

Prove $L = \{1^n 0^n \mid n \geqslant 0\}$ is not a regular language

sup. L is regular → pumping Lemma applies — pumping length p ($\geqslant 1$)

$w = 1^p 0^p$ $\quad w \in L$ $\quad |w| \geqslant p$ $\quad 2p \geqslant p$

$w = \underbrace{1 \cdots 1}_{p} \underbrace{0 \cdots 0}_{p}$

$w = xyz$

$|xy| \leqslant p$ $\quad |y| \geqslant 1$

$|xy| \leqslant p$ in $\Longrightarrow$ all valid decompositions (pumping Lemma)

xy only contains "1"s!

$|y| \geqslant 1$ $\Longrightarrow$ in all valid decompositions y must contain at least a single "1"

$xy^iz \longrightarrow$ generate strings with more "1"s than "0"s

$\notin L$